

MAGIC TOWER AS A DETERMINISTIC LONG-HORIZON PLANNING BENCHMARK: GO-EXPLORE, MCTS, AND POTENTIAL-BASED REWARD LEARNING

Bingsong Tong (2025310812), Zhizhou Qu (2025310268), Rui Chen (2025211105)
Deep Reinforcement Learning Course Project 2026

ABSTRACT

Magic Tower is a deterministic single-agent role-playing puzzle in which the agent must allocate health, attack, defense, keys, and money across many irreversible interactions. The first-ten-floor task studied here starts after the early thief event and terminates when the Floor-10 skeleton captain is defeated. The task is fully observable and rule-defined, but naive reinforcement learning rarely receives a useful terminal signal. We therefore separate three experimental lines: prompt-guided route discovery as diagnostic data, expert-trajectory potential learning from the hp403 route, and a warm-started AlphaZero-style policy/value line based on multiple successful routes. The hp403 trajectory is not used as a behavior-cloning target in the pure line; it is used only to learn a potential function for shaping $\gamma\Phi(s') - \Phi(s)$. The newest 4090 experiments show that direct root PUCT remains insufficient, but stage-boundary PUCT succeeds on several subproblems, including Floor-10 resources and the final captain. The remaining failure is stage composition: a locally successful low-gem route can still consume the yellow-key buffer required by later Floor-8 and Floor-10 resources.

1 INTRODUCTION

Magic Tower differs from many common game benchmarks because its difficulty is not caused by hidden information, stochasticity, or real-time control. The environment is deterministic, fully observable, and compact enough to simulate exactly. Its hardness comes from long-horizon resource dependence: a key spent on an unnecessary door, a potion taken before a shield, or one point of attack missing at a combat threshold can make a much later state infeasible. The project objective is to solve the post-thief first-ten-floor scenario and set the terminal flag

$$10f_captain_defeated(s) = 1$$

with positive health under strict replay. The simplified scenario removes the Floor-4 shop and flying item, keeps the Floor-6 and Floor-7 key merchants, and requires the real Floor-10 trap before the skeleton captain can be attacked.

The current result is not yet a fully learned solver. A prompt-guided diagnostic route reaches the boss with 403 HP and final attributes $(ATK, DEF) = (27, 27)$. The best self-generated pure route family reaches early and mid-stage milestones such as the sword and shield, and separate 4090 trials have reached the first Floor-10 resource, but they still fail from key and HP bottlenecks. This motivates a more principled hard-exploration method: Go-Explore stores stepping-stone states, while PUCT-MCTS performs local planning from each archived state. A learned potential from hp403 is allowed only as a shaping signal and is evaluated separately from the pure no-expert line.

2 MDP FORMULATION

We model the task as a finite-horizon deterministic single-agent MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}_{\text{valid}}, f, \mathcal{G}, \mathcal{F}, H, r).$$

A state $s \in \mathcal{S}$ contains the complete map state of Floors 1–10, the current floor and coordinate, hero resources ($HP, ATK, DEF, money$), key counts, consumed objects, triggered events, and flags. The valid action set is the set of currently executable macro-actions:

$$a_t \in \mathcal{A}_{\text{valid}}(s_t).$$

Each action chooses an interaction node, not an individual arrow key. The simulator expands it into a shortest path to a reachable item, stair, event, merchant, openable door, or killable enemy. The transition is deterministic:

$$s_{t+1} = f(s_t, a_t).$$

Terminal success is

$$\mathcal{G} = \{s : \text{flag}_{10F_captain}(s) = 1 \wedge HP(s) > 0\}.$$

Failures include death, invalid transitions, timeouts, and dead states with no useful executable actions. The sparse task reward is

$$r_{\text{task}}(s, a, s') = R_G \mathbf{1}[s' \in \mathcal{G}] - R_F \mathbf{1}[s' \in \mathcal{F}] - \kappa,$$

where κ is a small macro-step cost. The optimal value satisfies the single-player Bellman equation

$$V^*(s) = \max_{a \in \mathcal{A}_{\text{valid}}(s)} [r(s, a, f(s, a)) + \gamma V^*(f(s, a))].$$

Unlike a two-player zero-sum game, there is no minimax sign flip. A PUCT-MCTS rollout therefore backs up

$$G_k = r_k + \gamma G_{k+1} = \sum_{i=k}^{L-1} \gamma^{i-k} r_i + \gamma^{L-k} \hat{V}(s_L).$$

Combat is the main discontinuity. For a monster m , the damage formula is

$$D(s, m) = \left\lceil \frac{HP_m}{\max(1, ATK(s) - DEF_m)} \right\rceil \max(0, ATK_m - DEF(s)).$$

Thus a single red or blue gem can reduce future damage by hundreds of HP, while the same gem may have no effect away from a threshold. This makes dense reward design unusually sensitive.

3 ENVIRONMENT AND MACRO-ACTION ABSTRACTION

The simulator is implemented in Python and uses the extracted HTML5 data as the truth source. It supports strict battle damage, item pickup, doors, stair movement, Floor-6 and Floor-7 merchants, the Floor-8 special door, the Floor-10 trap, and the Floor-10 boss flag. The visualizer remains a separate validation tool; core learning code runs through `src/mota.env` and `src/mota.solver`.

The current action enumeration first computes the connected open region of the current floor. Previously, the implementation reconstructed a full path for every reachable cell on every state expansion. Since MCTS expands many states, this made the system CPU-bound. The new implementation stores only a BFS parent map and reconstructs paths lazily for selected semantic actions. More formally, let $R(s)$ be the set of grid cells reachable without passing through a door, enemy, stair, or NPC. The BFS stores

$$\text{parent}_s(x, y) = (x', y', d)$$

for each $(x, y) \in R(s)$. A macro-action is then generated from either a reachable interactive cell or an adjacent blocker:

$$\mathcal{A}_{\text{valid}}(s) = \{a_{\text{item}}, a_{\text{stair}}, a_{\text{event}}, a_{\text{npc}}, a_{\text{door}}, a_{\text{enemy}}\}.$$

The path $\tau(a)$ is recovered only when an action is emitted or executed. This preserves the old action set but removes most unused path-list allocations. Unit tests now verify that the parent-map reachability is equivalent to the previous path dictionary and that every lazily produced macro-action is executable under strict simulation.

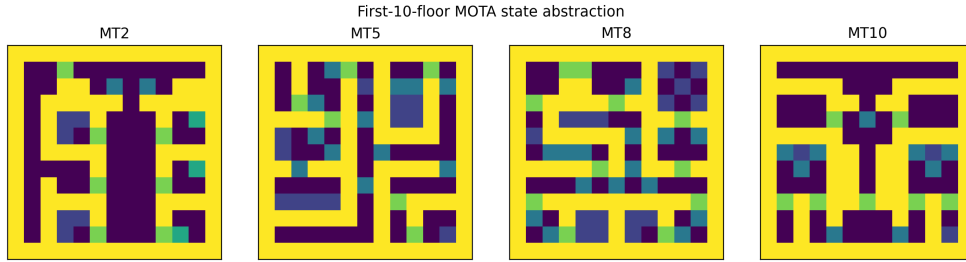


Figure 1: Representative floors in the simplified first-ten-floor scenario. The learning environment uses symbolic grid state rather than pixels.

4 WHY MAGIC TOWER IS HARD

Magic Tower is close to several established benchmarks but does not reduce to any one of them. Compared with AlphaZero-style Go or chess (Silver et al., 2017; 2018), every decision layer is a single-agent max layer, and random play almost never reaches an informative terminal signal. Board games always end in a win/loss label; Magic Tower usually enters a dead end before the boss is observable.

Compared with Sokoban, Magic Tower has irreversible spatial constraints plus resource variables. Sokoban deadlocks are mainly geometric. Magic Tower deadlocks are often numerical: the agent can legally open a door or kill a monster but thereby lose a key or health buffer required much later. Compared with Montezuma’s Revenge and other hard-exploration Atari tasks, Magic Tower has known symbolic rules, but exploration is blocked by exact resource thresholds rather than hidden rooms alone; this makes Go-Explore particularly relevant (Ecoffet et al., 2019). Compared with NetHack and MiniHack (Küttler et al., 2020), Magic Tower is less stochastic and more fully observable, but its deterministic damage thresholds create sharper local discontinuities.

ARC-AGI is also informative. The original ARC proposal emphasized abstract reasoning under small training sets (Chollet, 2019); ARC Prize 2024 reported substantial gains from program synthesis and test-time adaptation (Chollet et al., 2024); ARC-AGI-2 increased the difficulty for frontier systems (Chollet et al., 2025); ARC-AGI-3 reframed ARC into interactive turn-based environments (ARC Prize Foundation, 2026). Recent ARC-AGI-3 work also studies executable Python world models that are checked against observed transitions before planning (Rodionov, 2026). Magic Tower naturally extends this direction: it is an interactive symbolic environment with explicit state transitions, long-horizon planning, video-renderable trajectories, textual descriptions, and a large community corpus of demos and play episodes. These properties make it a promising benchmark for LLM agents that must infer rules, plan over resources, and justify actions.

5 THREE EXPERIMENTAL LINES

5.1 LINE 1: PROMPT-GUIDED ROUTE DISCOVERY

The first line uses Codex-style prompt-guided exploration as diagnostic route construction. This is not counted as an autonomous RL baseline. It was useful for finding simulator bugs, validating event logic, and producing a strict benchmark trajectory. The current hp403 route defeats the Floor-10 captain with

$$HP = 403, \quad ATK = 27, \quad DEF = 27, \quad money = 262.$$

It uses no Floor-4 shop and no flying item. The route is retained as a benchmark and for visual replay, not as evidence that the pure RL system has solved the task.

5.2 LINE 2: EXPERT-TRAJECTORY POTENTIAL LEARNING

The hp403 route is too narrow to define the full optimal policy. Direct behavior cloning from one route risks overfitting to one key order and one resource schedule. We therefore use it only to learn

a potential function $\Phi_\phi(s)$, following the policy-invariance motivation of potential-based reward shaping (Ng et al., 1999). For a successful trajectory $\tau = (s_0, a_0, \dots, s_T)$, we create progress and ranking labels:

$$y_t = t/T, \quad s_j \succ s_i \quad \text{if } j > i.$$

The potential loss combines regression, pairwise ranking, and dead-end separation:

$$\mathcal{L}_\Phi(\phi) = \sum_t (\Phi_\phi(s_t) - y_t)^2 - \alpha \sum_{j>i} \log \sigma(\Phi_\phi(s_j) - \Phi_\phi(s_i)) - \beta \sum_{(s^+, s^-)} \log \sigma(\Phi_\phi(s^+) - \Phi_\phi(s^-)).$$

The search reward is then potential-based:

$$r_{\text{search}}(s, a, s') = r_{\text{task}}(s, a, s') + \lambda (\gamma \Phi_\phi(s') - \Phi_\phi(s)).$$

This keeps task evaluation separate from shaping. Final success is always measured by strict replay and the boss flag, not by shaped reward.

In the current implementation we use a linear potential over hand-audited semantic factors rather than a free-form neural reward model. Let $c_k(s; g)$ denote the k -th stage-conditioned component for stage g , including hero resources, key pressure, reachable resources, blocked resource pressure, monster damage, marginal damage decrease under $ATK + 1$ or $DEF + 1$, equipment progress, Floor-10 resource progress, and boss margin. The learned potential is

$$\Phi_\phi(s, g) = \sum_k w_{g,k} c_k(s; g) + \sum_k w_k^{\text{global}} c_k(s; g).$$

Training examples are generated without asking the model to imitate the expert action. At route step t , the simulator enumerates all legal macro-actions $\mathcal{A}_{\text{valid}}(s_t)$. The hp403 action a_t^E is treated as a positive ranking candidate only for the one-step potential difference:

$$\Delta_k^E = \gamma c_k(f(s_t, a_t^E); g_t) - c_k(s_t; g_t),$$

and each valid alternative a^- provides

$$\Delta_k^- = \gamma c_k(f(s_t, a^-); g_t) - c_k(s_t; g_t).$$

The ranker optimizes

$$\mathcal{L}_{\text{rank}} = \sum_{t, a^-} \text{softplus} \left(- \sum_k w_{g_t, k} (\Delta_k^E - \Delta_k^-) \right) + \eta \|w\|_2^2.$$

The output is therefore a stage-conditioned potential, not a policy prior. The downstream AlphaZero-style and archive-MCTS experiments use hp403 only through

$$r_\Phi(s, a, s') = \gamma \Phi_\phi(s', g) - \Phi_\phi(s, g),$$

which enters single-agent MCTS backups as an edge reward and value-return target. No hp403 prefix route, behavior-cloning loss, or expert action prior is used in the potential-only run.

5.3 LINE 3: GO-EXPLORE ARCHIVE WITH PUCT-MCTS

The new pure exploration line combines a global archive with local MCTS. A cell abstraction maps states to coarse stepping stones:

$$\chi(s) = (\text{stage}, \text{floor}, \text{ATK}_b, \text{DEF}_b, \text{key}_b, \text{HP}_b, \text{mt10_progress}, \text{boss_margin}_b).$$

The archive stores top- K non-identical candidates per cell:

$$\mathcal{B}[\chi(s)] = \{(s_i, \tau_i, \text{score}_i)\}_{i=1}^K.$$

At each iteration, a cell is sampled according to novelty, progress, and visit penalty. From the archived state s_c , PUCT-MCTS selects a local action:

$$a = \arg \max_a \left[Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right].$$

For the pure line P is uniform or learned from self-generated search traces, not from hp403. In the potential-shaping line, hp403 influences only the edge reward and leaf value through Φ_ϕ . The archive score is lexicographic in success, stage progress, HP, keys, and path length, so a high shaping score cannot replace strict success.

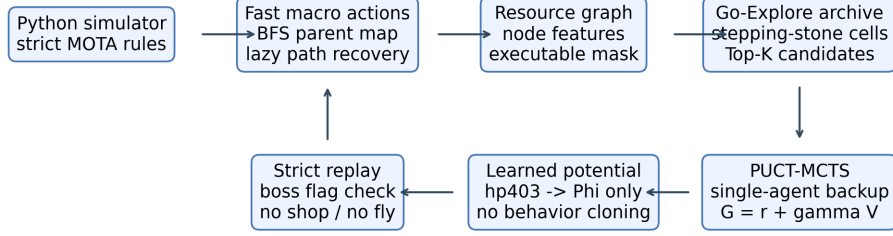


Figure 2: System organization for the next experiment. The visualizer is used only for route validation; the learning loop uses the Python simulator and resource-graph state.

6 CURRENT EXPERIMENTS

Codex-guided route. The diagnostic route reaches the captain and is used to test strict replay, visualization, no-shop/no-fly constraints, and reward curves. Figure 3 shows that shaped reward increases most strongly around equipment, gem, red-key, Floor-10, and boss events. This route family remains a diagnostic artifact rather than an autonomous RL baseline. The best strict route currently synced for visualization reaches the Floor-10 captain with positive HP and final resources around $(ATK, DEF) = (27, 27)$. It is used to debug route playback and reward factors, not to make claims about the pure search line.

Potential-only reward learning. The potential-learning run 4090_hp403_potential_only_20260602.014602 uses hp403 only through `scripts/learn_reward_from_hp403.py`. The learner produced 1578 pairwise action-ranking examples from 317 strict replay states. Under the learned one-step potential ranking, the hp403 action was top-1 in 52.1% of states and top-3 in 85.8%, with mean rank 2.00. This is intentionally not a perfect imitation signal. The learned potential is useful as a dense stage-progress preference, but final evaluation remains strict replay:

$$\text{Eval}(\tau) = \mathbf{1}[\text{flag}_{10F_captain}(s_T) = 1 \wedge HP(s_T) > 0].$$

AlphaZero-style policy/value model. The latest 4090 run, 4090_stage_warmstart_20260603.192636, trains graph policy/value models from multiple successful routes. This is a separate warm-start line, not the pure no-expert line. The network is a Transformer over the fixed resource-graph state:

$$f_{\theta}(G_s, g) = (p_{\theta}(\cdot \mid s, g), v_{\theta}(s, g)),$$

where G_s contains hero, item, enemy, door, stair, event, and global tokens, and g is the target stage. We use $d_{\text{model}} = 512$, eight heads, eight encoder layers, batch size 96, Adam learning rate $1.5 \cdot 10^{-4}$, and a policy/value loss

$$\mathcal{L}(\theta) = -\pi^{\top} \log p_{\theta}(\cdot \mid s, g) + 0.35 (G - v_{\theta}(s, g))^2 + 10^{-4} \|\theta\|_2^2.$$

Each model checkpoint is approximately 104 MB. Training uses 4090D GPUs; MCTS evaluation remains CPU-bound because every simulation still performs state cloning, legal macro-action enumeration, and simulator transitions.

Target stage	best loss	main evaluation	result
shield	0.021	root PUCT	failed, MT8, $HP = 103$
all_gems	0.063	root PUCT	failed, MT1, $HP = 428$
mt10_resources	0.048	root PUCT	failed, MT5, $HP = 754$
boss	0.069	root PUCT	failed, MT1, $HP = 564$
mid_gems	0.041	boundary PUCT	success
low_gems	0.094	boundary PUCT	success
mt8_gems	0.061	boundary PUCT	success

The table shows the main empirical conclusion. Low supervised loss is not enough for root planning. Direct root PUCT still burns keys and loops around the early tower. However, the same architecture succeeds when evaluated at stage boundaries. This means the model has learned useful local priors, but the global task needs hierarchical composition and archive return.

Go-Explore and root archive-MCTS. We also evaluated warm-start policy checkpoints inside archive-MCTS with `--protocol warmstart_policy`. From the initial state, the archive still failed to solve the task. The best warm-start archive states were:

shield : $MT8, HP = 233, ATK = 22, DEF = 11, YK = 2,$
 mt10_resources : $MT5, HP = 754, ATK = 20, DEF = 10, YK = 1, BK = 1,$
 boss : $MT5, HP = 680, ATK = 21, DEF = 10, YK = 1, BK = 1.$

These are better than random exploration but far from a complete route. The failure mode is consistent across root PUCT and root archive-MCTS: early-stage policy priors overvalue immediate combat, door opening, and local floor progress, while undervaluing the key buffer and shield-before-combat schedule needed by later floors.

Stage-boundary PUCT succeeds. The same checkpoints perform much better when the state is restored to a meaningful stage boundary, which is exactly the motivation for Go-Explore. We use successful-route prefixes only as a warm-start diagnostic. At each boundary the model and MCTS must choose the suffix actions themselves under strict HP. The results are:

Experiment	prefix	macro suffix	HP	(ATK, DEF)	success
mid_gems	76	24	294	(23, 22)	yes
low_gems	100	20	303	(25, 24)	yes
mt8_gems	147	15	245	(26, 25)	yes
mt8_gems	155	12	172	(26, 26)	yes
mt10_resources	236	2	915	(27, 27)	yes
boss	251	32	247	(27, 27)	yes
boss	283	8	436	(27, 27)	yes

These successes are not a full autonomous solution, because the prefix is provided. They are nevertheless important: they show that stage-conditioned PUCT can solve the suffix subproblems once the archive returns the agent to a good stepping stone.

Stage composition failure. We then attempted to compose learned stages rather than replaying each original prefix. The chain

`mid_gems` $\rightarrow$ `low_gems`

succeeded and ended at

$(MT1, HP = 303, ATK = 25, DEF = 24, YK = 2, BK = 0).$

However, continuing from this generated state into `mt8_gems` failed:

$(MT8, HP = 193, ATK = 25, DEF = 24, YK = 0, BK = 1).$

The route completed a local low-gem objective but spent the yellow-key buffer required by the next stage. This is the central open problem at the current checkpoint. We therefore started a stronger middle-stage training set with targets `lower_gems`, `mt8_hp_ready`, and `pre_mt10_buffer`. These targets make the value function care about continuation viability rather than only local resource completion.

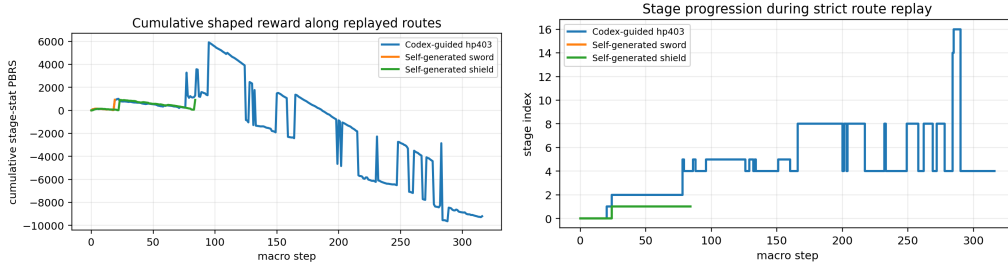


Figure 3: Left: cumulative stage-stat PBRS along replayed route families. Right: stage progression under strict route replay. The hp403 curve is diagnostic, not pure-RL supervision.

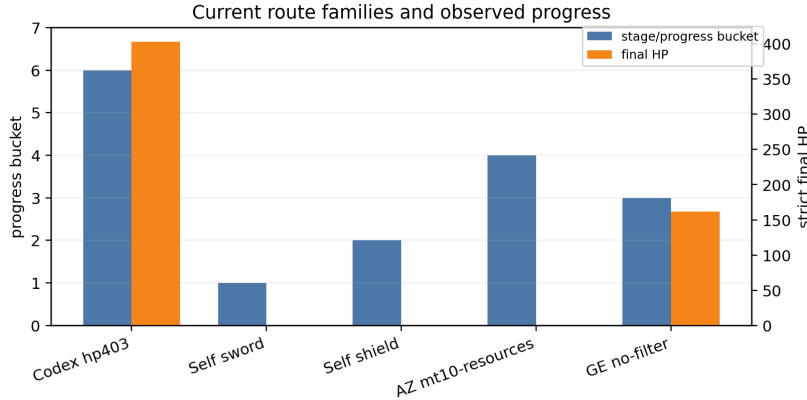


Figure 4: Current route families. Pure routes have reached important milestones but still fail before the final captain.

CPU-bound diagnosis. During policy training, all four 4090D GPUs reach high utilization and use about 8.7 GB each. During PUCT and archive-MCTS, GPU usage falls to roughly 645 MB with near-zero utilization, while each process consumes one CPU core. This confirms that the bottleneck is not neural inference capacity. It is the simulator side: macro-action enumeration, BFS reachability, state cloning, and per-child legality checks dominate rollout time. The next engineering step is to batch graph inference and reduce simulator overhead through transposition tables, compact consumed-node bitsets, reachability caches keyed by dynamic blockers, and lazy path recovery.

7 NEXT 4090 EXPERIMENT

All heavy experiments now run on the 4090 pod; local execution is limited to editing, report compilation, route visualization, and light tests. We keep three result labels:

`pure_search_rl`, `hp403_potential`, `warmstart_policy`.

The first label must not use hp403 or other successful routes. The second may use hp403 only through Φ_ϕ . The third may use successful routes to train a policy/value model, but it is reported as an expert-aided comparison.

The immediate run plan is:

Stage	Training target	Evaluation	Success criterion
Middle resources	<code>lower_gems</code>	PUCT from generated prefix	preserves YK buffer
HP buffer	<code>mt8_hp_ready</code>	PUCT from generated prefix	enough HP before Floor 8
Pre-10F buffer	<code>pre_mt10_buffer</code>	PUCT/archive-MCTS	enough YK for Floor 10
Final suffix	<code>mt10_resources</code> , boss	strict replay	captain flag true

The main question is whether a generated state, not a replayed expert prefix, can be passed from one stage policy to the next:

$$s_0 \xrightarrow{\pi_{\text{mid}}} s_1 \xrightarrow{\pi_{\text{lower}}} s_2 \xrightarrow{\pi_{\text{mt8}}} s_3 \xrightarrow{\pi_{\text{mt10}}} s_4 \xrightarrow{\pi_{\text{boss}}} s_T \in \mathcal{G}.$$

If this chain fails because a stage greedily spends keys or HP, the next model will train on continuation-aware values:

$$\tilde{G}_t = G_t + \lambda_1 \mathbf{1}[YK \geq 2] + \lambda_2 \mathbf{1}[HP \geq h_g] + \lambda_3 \Delta \text{BossMargin} - \lambda_4 \mathbf{1}[\text{dead_end}].$$

This is not a replacement for task reward. It is a training target for stage-value estimation and archive cell priority.

The CPU-bound engineering plan is also part of the next run. PUCT currently performs one network inference per expanded state, but most wall-clock time is spent before inference. We will therefore:

1. cache legal macro-actions by compact dynamic-blocker keys rather than full scalar state;
2. represent consumed nodes with bitsets and avoid full floor-grid clones;
3. use a transposition table for repeated MCTS states;
4. batch neural evaluations across leaf states;
5. reconstruct primitive paths only for the final selected macro-action.

The target is not merely higher GPU utilization. The target is more archive expansions per hour under strict replay, because the current failure is insufficient diverse exploration of feasible key schedules.

8 FUTURE WORK: MOTA AS AN LLM BENCHMARK

Magic Tower can be turned into a benchmark suite beyond this first-ten-floor RL project. The environment can expose three synchronized interfaces: symbolic JSON state, natural-language state summaries, and video frames from the visualizer. An LLM agent can then be evaluated on action selection, trajectory-level planning, rule discovery, and post-hoc explanation. This is analogous to ARC-AGI’s emphasis on test-time reasoning, but Magic Tower adds an explicit transition function and long interactive trajectories. The community already contains many game variants, demos, and complete episodes; these can support supervised route modeling, offline RL, reward learning, and held-out map generalization. A robust benchmark should separate seen maps, unseen maps, seen rule sets with shuffled layouts, and video-only or text-only observation conditions.

9 LIMITATIONS

The current simulator is a first-ten-floor research subset, not a full 50-floor solver. Macro-actions hide primitive movement, which is appropriate for resource planning but not for end-to-end pixel control. The hp403 route is a single expert trajectory and therefore insufficient for imitation learning by itself. Finally, manual prompt-guided exploration helped diagnose the environment; the next experiments must keep this diagnostic data separate from pure search/RL results.

10 CONCLUSION

Magic Tower is best treated as a deterministic long-horizon planning benchmark with sparse terminal reward and discontinuous resource thresholds. Pure AlphaZero-style root search is not enough because useful terminal states are too rare and early local choices can destroy late resource feasibility. The newest 4090 experiments sharpen this conclusion: stage-conditioned policy/value models can solve suffix subproblems from good archive states, but naive composition still fails when a locally successful stage spends the yellow-key or HP buffer needed by the next stage. The next system therefore combines Go-Explore’s archive of stepping stones, continuation-aware stage values, single-agent PUCT-MCTS backups, and potential-based shaping learned from the hp403 route. This preserves a clean distinction between pure no-expert search and expert-aided shaping while giving the project a more appropriate mechanism for discovering complete first-ten-floor routes.

REFERENCES

- ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026.
- François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- Francois Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report. *arXiv preprint arXiv:2412.04604*, 2024.
- Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- Heinrich Küttler, Nantas Nardelli, Alexander Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktäschel. The nethack learning environment. *NeurIPS*, 2020.
- Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, 1999.
- Sergey Rodionov. Executable world models for ARC-AGI-3. *arXiv preprint arXiv:2605.05138*, 2026.
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.